COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE21S2 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	14/10/2026	**DATE SUBMITTED:**	16/10/2026
NAME:	Lopez, Andrei Dion Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Binary Search Tree Construction	The binary search tree construction follows the algorithm: when the value is smaller, it goes to the left, and when the value is larger, it goes to the right. Thus, satisfying the recursion strategy due to its repeating nature.
Brute Force	Linear Search	Linear search goes through each node one-by-one to search for the value it was told to find. This algorithm still searches the entire node even if that value does not exist within the actual list itself, thus satisfying the brute force strategy
Backtracking	Depth Tree Search	Depth tree search satisfies the algorithm of backtracking due to its three types, which are pre-order, in-order, and post-order. These three aggressively go down a single path before backtracking for the other nodes.
Greedy	Selection Sort	The selection sort assumes that the starting position in the node is the highest value before sorting the algorithm as it finds the actual smallest value. This satisfies the greedy strategy simply because it is not the most optimal solution.
Divide-and-conquer	Merge Sort	The merge sort algorithm continuously halves the list/array into separate nodes until they are on their own, which satisfies the action of divide-and-conquer as it fixes the list.

Table 12-1. Algorithmic Strategies and Examples

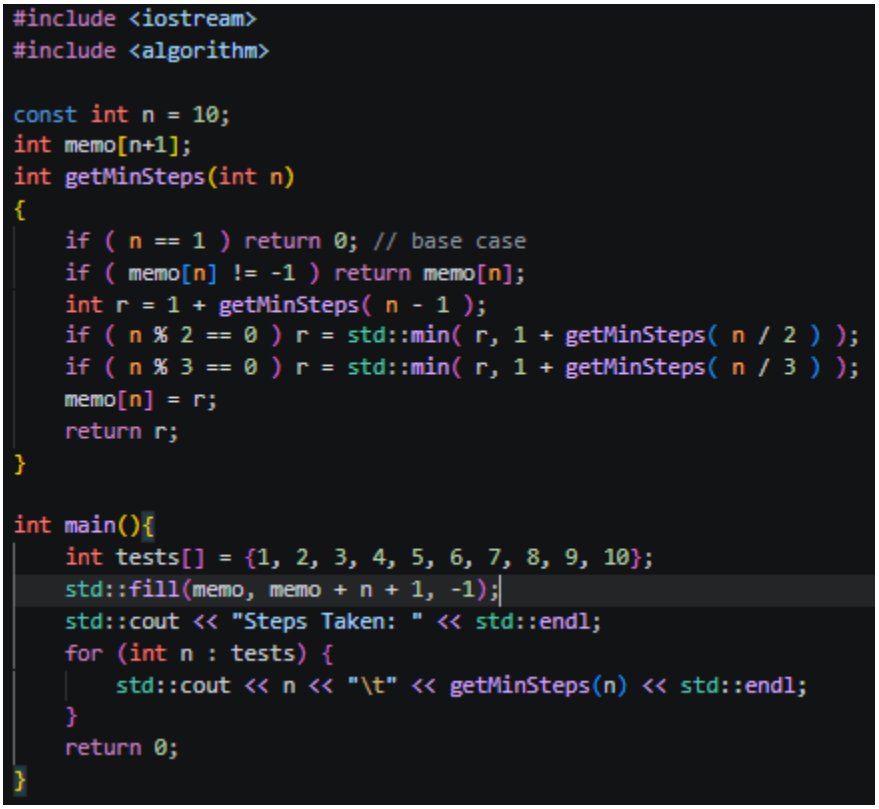
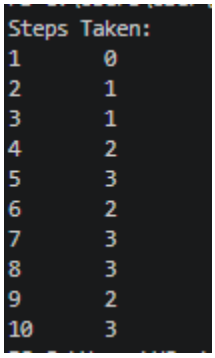
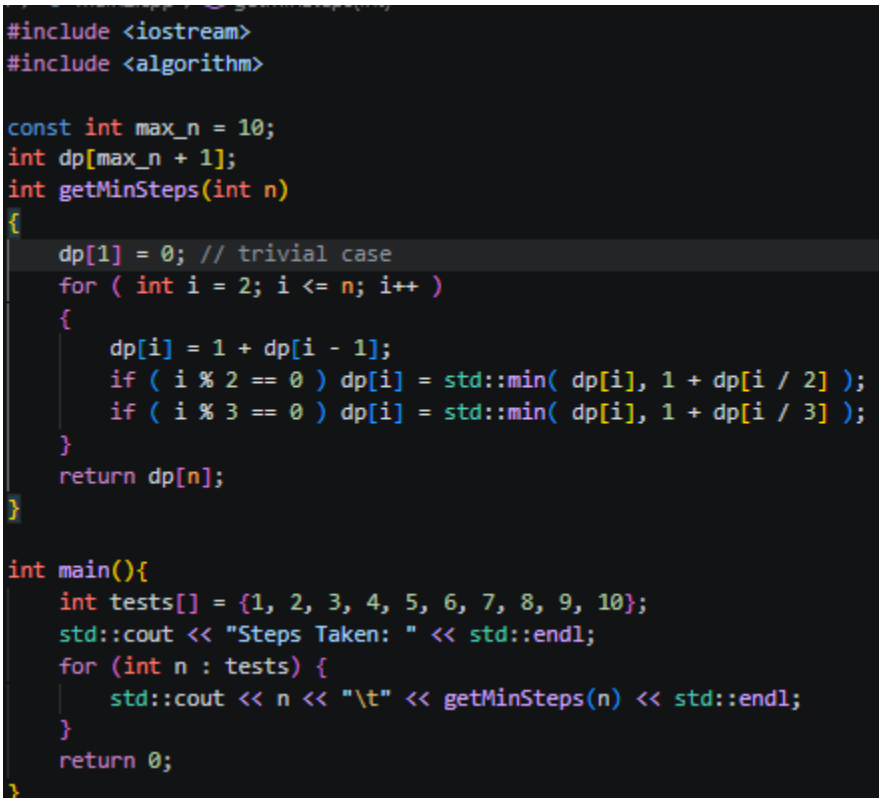
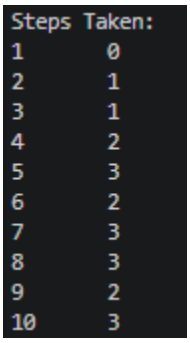
Screenshot	 
Analysis	When I approached the problem and set it up in VS Code, the main issue was that n was not identified. Another problem arose: simply trying int max_n also resulted in an error, so I used a const, and it eventually worked in main. I also added std::fill to mark everything as not solved yet.

Table 12-2. Memoization Implementation

Screenshot	 
Analysis	The bottom-up dynamic programming produced the same output, and the same solution was used for the missing identifier n. In the bottom-up implementation, there was no memo, so it didn't have to check for “already solved.” The main difference from this is that the for-loop does all the work starting at 1, where the known answer is 0, and slowly moves up one number at a time.

```cpp
#include <iostream>
#include <algorithm>

const int max_n = 10;
int dp[max_n + 1];
int getMinSteps(int n)
{
    dp[1] = 0; // trivial case
    for ( int i = 2; i <= n; i++ )
    {
        dp[i] = 1 + dp[i - 1];
        if ( i % 2 == 0 ) dp[i] = std::min( dp[i], 1 + dp[i / 2] );
        if ( i % 3 == 0 ) dp[i] = std::min( dp[i], 1 + dp[i / 3] );
    }
    return dp[n];
}

int main(){
    int tests[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    std::cout << "Steps Taken: " << std::endl;
    for (int n : tests) {
        std::cout << n << "\t" << getMinSteps(n) << std::endl;
    }
    return 0;
}
```

```
Steps Taken:
1       0
2       1
3       1
4       2
5       3
6       2
7       3
8       3
9       2
10      3
```

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

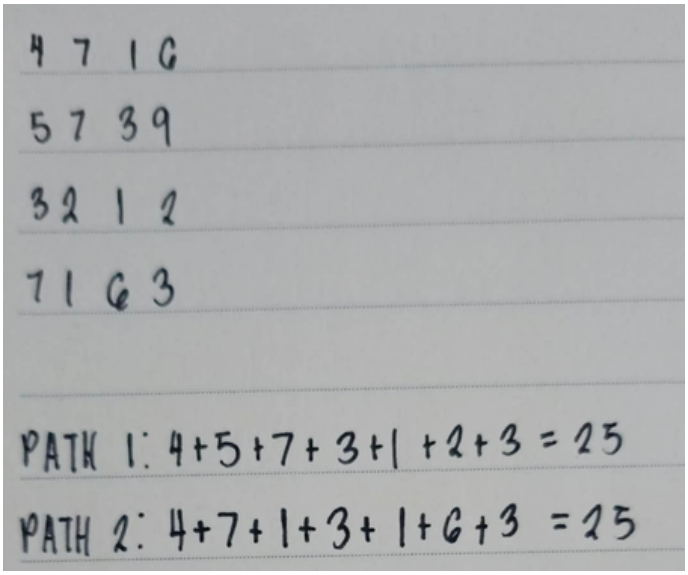


Figure 1. Problem Solving by Hand

```cpp
#include <iostream>

const int M = 4, N = 4;

int countPaths(int mat[M][N], int cost) {
    int dp[M][N][100] = {};
    if (mat[0][0] > cost) return 0;
    dp[0][0][mat[0][0]] = 1;

    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            for (int c = 0; c <= cost; c++) {
                if (i == 0 && j == 0) continue;
                if (c < mat[i][j]) continue;
                int before = c - mat[i][j];
                if (i > 0) dp[i][j][c] += dp[i-1][j][before];
                if (j > 0) dp[i][j][c] += dp[i][j-1][before];
            }
    return dp[M-1][N-1][cost];
}

int main() {
    int a[M][N] = {{4,7,1,6},{6,7,3,9},{3,8,1,2},{7,1,7,3}};
    std::cout << "Printed matrix, cost 26: " << countPaths(a, 26) << std::endl;

    int b[M][N] = {{4,7,1,6},{5,7,3,9},{3,2,1,2},{7,1,6,3}};
    std::cout << "Original matrix, cost 25: " << countPaths(b, 25) << std::endl;
}
```

Figure 2. Working c++ code

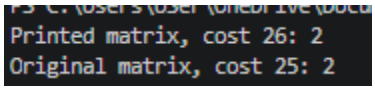


Figure 3. Snippet of the output in the terminal

ANALYSIS	
LOPEZ	The program itself utilized a dynamic programming approach from the bottom-up. It starts out with figuring out the total size of the matrix using the set variables to 4 for both M and N, which shows us that the matrix achieved the size of 4x4. We proceed to making a table for the variables to be indicated and stored. Continuing from there, we start filling up the matrixes from the [0,0] coordinate where in the "for loop and if" statements will serve for filling in the values. Afterwards, we then move the cells above to the cells on the left which shifts the current cell's cost. Resulting in the program building and tallying every cell and their ways to get to their potential paths. This is possible by adding the tallies it records from every cell it moves.
REALUBIT	The program runs with the bottom-up approach of dynamic programming. Firstly, initialize the total size of the matrix using the variables set to 4 for both M and N, allowing us to have a size of a matrix of size 4x4. We then make a table for where the values will be stored. Next, we make sure the filling of the matrix starts at the coordinate [0,0], where the for loop and if statements will be responsible for filling in the values. Once filled, we return the cost of going from the cell above to the cell on the left, which is shifted by the current cell's cost. Essentially, the program builds up for every cell and tallies how many ways to get there with each possible path. It does this by adding the tallies of every cell it moves from.
CONCLUSION	
LOPEZ	In this activity we discovered different algorithm strategies that are utilized despite obvious methods. Some of these methods can include backtracking and divide-and-conquer, which is mainly used for tree navigation and sorting algorithms. Another example of these methods is the procedure and supplementary materials, which recalled the previous algorithms that were used during the course. Despite our findings, there was some difficulty when it came to the supplementary material, but we utilized iteration instead of recursion to help with the procedure. I would like to think we did well for this activity.
REALUBIT	In this hands-on activity, we discussed the different algorithm strategies that were not obvious at first but were already being used unconsciously; examples of these were backtracking and divide-and-conquer, which were primarily used for tree navigation and sorting algorithms like merge sort, selection sort, and quick sort. Following the discussion are the procedure and supplementary materials, which consisted of recalling some of the algorithms that were previously used in the course and which can be considered for that strategy. The supplementary material itself was slightly difficult, without using the bottom-up approach earlier in the procedure, where we use iteration instead of recursion, which slightly eased the difficulty. Overall, I would say that I did fine for the whole activity;

	albeit, there were still some difficulties that were encountered, like the persistent error from the procedure activities.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Concepts and Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The use of AI was strictly limited to the use of expounding on concepts such as dynamic programming, memoization, and the bottom-up approach. It also consisted of code assistance for errors that were not explained properly by the error lens in VS Code.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.